

TRABALHO PRÁTICO – UNIDADE 1

Orquestração e Implantação Local de Microserviços no Kubernetes com k3d e Helm

Estudante: Guilherme Stein

Disciplina: Orquestração de Containers & Kubernetes

1. Introdução e Arquitetura do Sistema

Este relatório apresenta a migração e orquestração do jogo demonstrativo *Guess Game* para o ambiente de orquestração de containers **Kubernetes**, utilizando ferramentas do ecossistema cloud-native como **k3d** (cluster K3s local baseado em Docker) e **Helm** como gerenciador de pacotes.

A arquitetura original do projeto baseada em Docker Compose foi reestruturada para seguir os padrões declarativos e resilientes do Kubernetes, dividida em três componentes principais altamente coesos e fracamente acoplados:

Componente	Função do Serviço	Recursos Kubernetes Utilizados
Database	Banco de dados relacional Postgres para armazenar dados de usuários e jogos.	Deployment, ClusterIP Service, PVC (Dynamic Path), Secret
Backend	API REST Flask (Python 3.12) executando lógica de negócios e validação de respostas.	Deployment, ClusterIP Service, HPA (Horizontal Pod Autoscaler)
Frontend	Cliente React servido por NGINX, que atua como proxy reverso para o Backend.	Deployment, NodePort Service, ConfigMap (Nginx Routing)

2. Considerações de Design Críticas (Dia 0)

2.1. Rede (Networking)

No Kubernetes, a comunicação entre Pods é baseada no modelo *IP-per-Pod*, gerenciada pelo Container Network Interface (CNI), que no k3d é o **Flannel**. Para isolar e estabilizar o tráfego interno:

- **Service Discovery:** A comunicação interna backend-db e frontend-backend ocorre exclusivamente via DNS interno. O Service do banco de dados é nomeado db e o do Flask é backend, eliminando IPs fixos.
- **Nginx DNS Resolution:** A configuração original do Docker Compose utilizava a diretiva de resolução interna de DNS do Docker (127.0.0.11). No Kubernetes, isso causaria erro 502. Um ConfigMap foi implementado para montar a configuração correta do NGINX no container frontend, apontando a rota /api/ diretamente para o Service ClusterIP http://backend:5000.
- **Exposição Externa:** Utilizou-se um serviço do tipo **NodePort** mapeado na porta estática **30080** no host local, eliminando a dependência obrigatória de Ingress Controllers em clusters de desenvolvimento.

2.2. Armazenamento (Storage)

A persistência dos dados do banco Postgres é garantida por meio de um **PersistentVolumeClaim (PVC)**. O Kubernetes local da distribuição K3s utiliza a StorageClass padrão **local-path**. Esse provisionador dinâmico aloca e monta diretórios do nó host de forma transparente, permitindo que os dados persistam mesmo após exclusões, reinicializações ou atualizações do Pod do banco de dados.

2.3. Disponibilidade e Escalabilidade (HPA)

A alta disponibilidade do backend Flask é controlada de forma dinâmica pelo recurso de **Horizontal Pod Autoscaler (HPA)**. Para que o HPA funcione no k3d, o componente **metrics-server** foi implantado no namespace kube-system com a flag `--kubelet-insecure-tls` para contornar problemas de certificados autoassinados em ambiente local.

Foram definidas as seguintes políticas de auto-scaling:

- **Replicas Mínimas:** 2 Pods para garantir tolerância a falhas básicas.
- **Replicas Máximas:** 5 Pods para conter uso excessivo de recursos no nó.
- **Métrica Alvo:** 50% de uso de CPU do Pod.
- **Resource Requests/Limits:** Definidos como 100m CPU / 128Mi RAM (Request) e 200m CPU / 256Mi RAM (Limit) no backend. Sem limites definidos, o HPA fica inativo, pois não tem base percentual para cálculo de escala.

2.4. Gestão de Permissões (RBAC)

Em ambientes produtivos, a segurança é regida pelo **Role-Based Access Control (RBAC)**. A aplicação é executada sob a ServiceAccount padrão do namespace, sem privilégios extras adicionais de ClusterAdmin. Caso o backend precisasse consultar a API do Kubernetes, criaríamos uma **Role** restrita com verbos de leitura associada via **RoleBinding** à ServiceAccount específica do backend, seguindo o Princípio do Menor Privilégio.

2.5. Facilidade de Implantação e Manutenibilidade

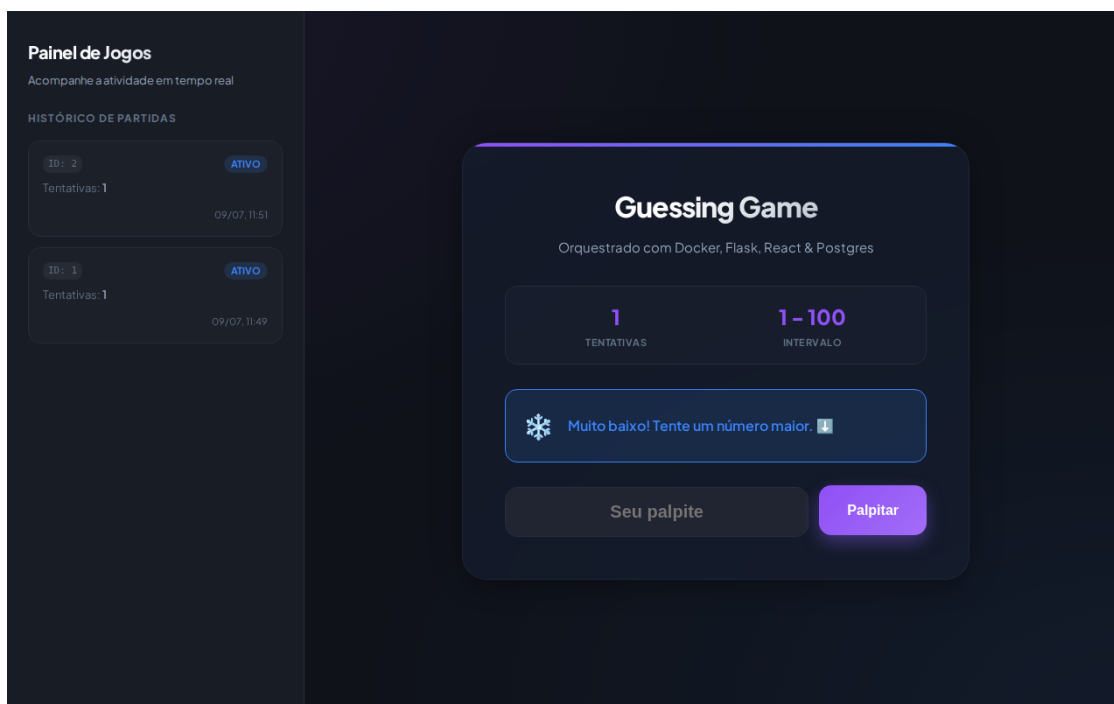
Utilizou-se o **Helm (Charts)** para estruturar a implantação como um pacote parametrizável. As definições de recursos, credenciais, imagens e portas foram centralizadas no arquivo `values.yaml`. Essa modularidade garante que a atualização das imagens (Dia 2) ocorra de forma simples e segura, utilizando o comando `helm upgrade` apenas alterando a tag da imagem no `values`.

3. Instalação e Implantação (Dia 1)

Para implantar a aplicação localmente no cluster k3d, execute o script automatizado `start_k8s.sh` na raiz do repositório. O fluxo do script executa as seguintes etapas:

1. Cria o cluster k3d mapeando a porta host 30080 para o NodePort do cluster.
2. Copia e importa as imagens Docker locais do backend e frontend diretamente para os nós do k3d.
3. Implanta e configura o Metrics Server com suporte a certificados insecure TLS.
4. Instala o Helm Chart 'guess-game' no namespace default.
5. Aguarda até que todos os componentes estejam totalmente prontos (Running/Ready).

Abaixo é exibido o screenshot do jogo rodando no navegador e integrado ao banco de dados via NodePort:



4. Operação e Manutenção do Cluster (Dia 2)

A fase de manutenção contínua e suporte do Kubernetes foca em estabilidade operacional:

- **Atualizações Zero-Downtime:** As atualizações de aplicação utilizam a estratégia padrão de *RollingUpdate*. Novos pods são criados e passam por Probes de inicialização antes que os pods antigos sejam removidos.
- **Health Checks Proativos:** A API Flask possui um endpoint de saúde (`/api/health`) monitorado por *readinessProbe* e *livenessProbe* que removem automaticamente pods lentos ou travados do balanceamento.
- **Políticas de Backup:** Para o banco Postgres local, o backup é feito via `pg_dump` direcionado ao volume NFS persistent ou diretamente via CronJobs do Kubernetes executando tarefas agendadas de dump.

5. Comparativo: On-premises vs. Cloud Provider

Ao migrar de um ambiente local (k3d/on-premises) para uma nuvem pública (AWS EKS, GCP GKE, Azure AKS), várias camadas de infraestrutura mudam significativamente para alavancar os serviços gerenciados:

Dimensão	Ambiente On-premises / Local	Nuvem Pública (EKS, GKE, AKS)
Rede	NodePort exposto em porta alta (ex: 30080). Balanceamento nativo do K8s.	Integração nativa com Cloud Load Balancer (NLB/ALB) via Service.
Armazenamento	Armazenamento local (local-path) preso aos nós. Risco de perda.	Provisão dinâmica via CSI integrando volumes EBS, Azure.
Disponibilidade	Limitada pelo hardware dos nós físicos locais.	Multi-AZ automático, autoscaling de nós (Cluster Autoscaler) e SLA.
Manutenção	Administração completa do cluster, SO, rede e atualizações de K8s.	Atualizações de K8s controlado pelo provedor. Atualizações simplificadas.

6. Referências Bibliográficas

1. BURNS, Brendan; BEDA, Joe; HIGHTOWER, Kelsey. *Kubernetes: Up & Running: Dive into the Future of Infrastructure*. 2. ed. Sebastopol: O'Reilly Media, 2019.
2. LUKSA, Marko. *Kubernetes in Action*. Shelter Island: Manning Publications, 2017.
3. HELM CONTRIBUTORS. *Helm Documentation*. Disponível em: <<https://helm.sh/docs/>>. Acesso em: 9 jul. 2026.
4. KUBERNETES AUTHORS. *Kubernetes Documentation*. Disponível em: <<https://kubernetes.io/docs/>>. Acesso em: 9 jul. 2026.